



2025 金三银四面面试突击早鸟课第二弹——字节T2-1岗位面试真题解析

0219.zip
46.08KB



往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)

字节T1-2/2-1的要求

T1-2：有执行力的工程师

能够理解需求，了解需求方整体业务的重点方向，高质量的完成交付。

编写的代码有合理的架构设计，具备良好的代码风格，合乎代码规范。

T2-1：能够独立承担任务的工程师

能够理解业务的核心目标与发展方向，能够独立完成中等难度项目的开发，有对应的项目管理能力与意识。

能够输出高质量的文档，参与团队的技术分享。可以作为实习生和应届毕业生的导师，帮助他们登陆和适应环境。

能够有自己的中长期的规划。

经典算法题

动态规划：接雨水（Hard）

i “接雨水”被戏称为字节的人都会的经典算法题，是 leetcode 中的第 42 题，等级为【Hard】，虽然在【Hard】中它是守门员级别，但实际上面试中如果碰到，还是一个比较有考验的题目。

在 leetcode 中还有该题目的升级版，“3D版接雨水”，即在普通一维数组的基础上，升级到二维数组，进行更多方向的大小值对比。

实际上，在字节前端岗位面试中，该类算法题已经是天花板级别。

给定 `n` 个非负整数表示每个宽度为 `1` 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。



示例1：

输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出：6

解释：上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例2：

输入：height = [4,2,0,3,2,5]

输出：9

知识点

动态规划

动态规划（Dynamic Programming，简称DP）是一种解决复杂问题的优化方法，它将原问题分解为一系列相互重叠的子问题，并通过存储子问题的解来避免重复计算。动态规划的核心思想是利用空间换时间，通过存储已经计算过的子问题的解，从而减少计算量。

动态规划的特点

- **重叠子问题**：原问题可以分解为若干个重叠的子问题。
- **最优子结构**：原问题的最优解可以通过其子问题的最优解构造出来。
- **无后效性**：某个状态一旦确定，不会受到后续决策的影响。

重叠子问题

动态规划的第一个特征是问题可以被分解成相似的子问题，并且这些子问题会被多次求解。例如，在计算斐波那契数列时，`F(n) = F(n-1) + F(n-2)`，`F(n-1)` 和 `F(n-2)` 会被多次计算。

最优子结构

动态规划的第二个特征是最优解包含了其子问题的最优解。如果一个问题最优解可以从其子问题的最优解有效地构造出来，那么该问题就具备最优子结构特性。

无后效性

无后效性有两层含义，第一层含义是，在推导后面阶段的状态的时候，我们只关心前面阶段的状态值，不关心这个状态是怎么一步一步推导出来的。第二层含义是，某阶段状态一旦确定，就不受之后阶段的决策影响。无后效性是一个非常“宽松”的要求。只要满足前面提到的动态规划问题模型，其实基本上都会满足无后效性。

实现方式

动态规划有两种主要实现方式：

- **自底向上**：也称为迭代法，先解决最简单的小规模问题，然后逐渐构建更大规模问题的解。
- **自顶向下**：也称为递归法+记忆化，从问题的大规模开始尝试解决，遇到子问题时递归求解，并将已解决的子问题的答案缓存起来供后续使用

动态规划的步骤

1. **识别状态**：确定问题的状态表示，也就是哪些信息是必须保存的
2. **定义状态转移方程**：找出从一个状态到另一个状态的转移方程，这通常涉及到递推关系。
3. **确定初始条件**：明确问题的边界条件或者起始状态。
4. **计算所有状态**：按照一定的顺序计算所有状态的值，通常是从最简单的状态开始，逐步构建到复杂的状态。
5. **根据计算结果得出最终答案**：根据计算过程中得到的所有状态值，最后得出整个问题的解。

动态规划的应用

动态规划广泛应用于各种问题，如：

- 最短路径问题（如 Dijkstra 算法）
- 背包问题
- 字符串匹配（如最长公共子序列）
- 硬币找零问题
- 编辑距离
- 路由选择

“动态规划算法”和”回溯算法”的对比

使用表格来对比“动态规划算法”和“回溯算法”的异同及适用场景可以使信息更加清晰明了。以下是一个详细的对比表格：

对比项	动态规划算法	回溯算法
定义	一种通过将问题分解成更小的子问题，并利用子问题的解来构建原问题的解的方法。	一种通过试探性地构建问题的解，并在发现解不可行时撤销之前的步骤的方法。
核心思想	最优化原理 + 重叠子问题 + 记忆化	试探性构建解 + 回溯 + 剪枝
子问题特点	子问题具有重叠性质	子问题通常是独立的
解的构建	通过记忆化避免重复计算	通过回溯撤销错误的选择
实现方式	自底向上（迭代法）或自顶向下（递归加记忆化）	递归 + 剪枝
时间复杂度	通常较低，因为通过记忆化减少了重复计算	较高，因为它需要探索所有可能的路径
空间复杂度	需要额外的空间来存储子问题的解	主要消耗栈空间来记录递归调用
适用场景	最优化问题（如最长公共子序列、最短路径、背包问题等）	搜索问题（如N皇后问题、迷宫问题等）
示例	背包问题	N皇后问题
优点	显著减少计算量，提高效率	可以探索所有可能性，找到所有可行解
缺点	需要额外空间来存储中间结果	时间复杂度高，不适合大规模问题

回溯算法是个“万金油”。基本上能用动态规划、贪心解决的问题，我们都可以用回溯算法解决。回溯算法相当于穷举搜索。穷举所有的情况，然后对比得到最优解。不过，回溯算法的时间复杂度非常高，是指数级别的，只能用来解决小规模数据的问题。对于大规模数据的问题，用回溯算法解决的执行效率就很低了。

尽管动态规划比回溯算法高效，但是，并不是所有问题，都可以用动态规划来解决。能用动态规划解决的问题，需要满足三个特征，最优子结构、无后效性和重复子问题。在重复子问题这一点上，动态规划和分治算法的区分非常明显。分治算法要求分割成的子问题，不能有重复子问题，而动态规划正好相反，动态规划之所以高效，就是因为回溯算法实现中存在大量的重复子问题。

贪心算法实际上是动态规划算法的一种特殊情况。它解决问题起来更加高效，代码实现也更加简洁。不过，它可以解决的问题也更加有限。它能解决的问题需要满足三个条件，最优子结构、无后效性和贪心选择性（这里我们不怎么强调重复子问题）。

其中，最优子结构、无后效性跟动态规划中的无异。“贪心选择性”的意思是，通过局部最优的选择，能产生全局的最优选择。每一个阶段，我们都选择当前看起来最优的决策，所有阶段的决策完成之后，最终由这些局部最优解构成全局最优解。

双指针

参考答案

暴力版

从最左侧开始，依次查找每个成员左侧和右侧的最大值，确定当前的接水量。

```
1  /**
2   * @param {number[]} height
3   * @return {number}
4   */
5  var trap = function(height) {
6      let sum = 0;
7      for (let i = 0; i <= height.length - 1; i++) {
8          if (i === 0 || i === height.length - 1) {
9              continue;
10         }
11         let currentHeight = height[i];
12         let leftHighest = 0;
13         let rightHighest = 0;
14         for (let j = i - 1; j >= 0; j--) {
15             if (height[j] > leftHighest) {
16                 leftHighest = height[j]
17             }
18         }
19         for (let k = i + 1; k <= height.length - 1; k++) {
20             if (height[k] > rightHighest) {
21                 rightHighest = height[k]
22             }
23         }
24         let lower = leftHighest > rightHighest ? rightHighest : leftHighest;
25         if (lower > currentHeight) {
26             sum += (lower - currentHeight);
27         }
28     }
29     return sum;
30 };
```

利用缓存

上面的方法，需要经历大量的重复计算，可以先进行两轮遍历，依次计算出每个成员左侧和右侧的最大值。每次计算时，只要对比左侧相信柱子高度与其对应的左侧最大值，即可快速获取最大值，右侧同理，只不过从最右的成员开始计算。

```
1  var trap = function(height) {
2      let sum = 0;
3      const leftMax = [0];
4      const rightMax = [];
5      rightMax[height.length - 1] = 0;
6      console.time('max')
7      for (let i = 1; i <= height.length - 2; i++) {
8          if (i === 1) {
9              leftMax[i] = height[0];
```

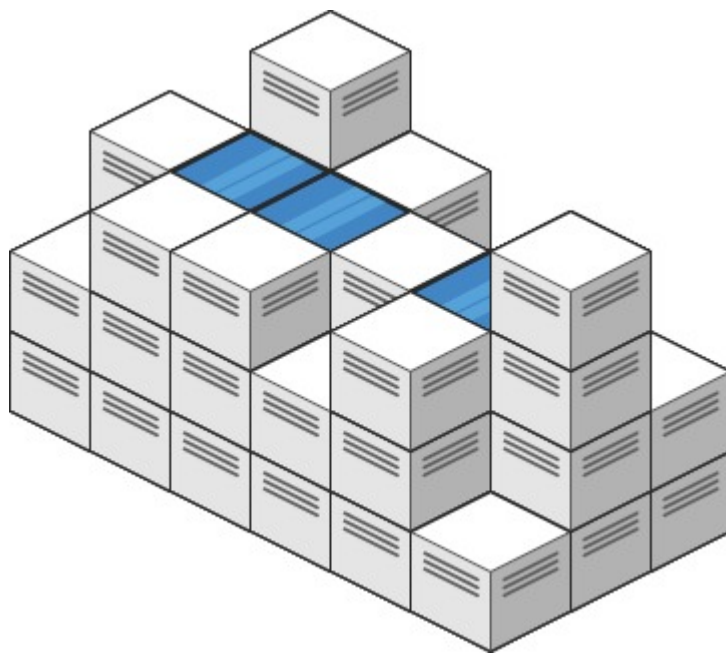
```

10         continue
11     }
12     leftMax[i] = Math.max(leftMax[i - 1], height[i - 1])
13 }
14 for(let i = height.length - 2; i >= 0; i--) {
15     if (i === height.length - 2) {
16         rightMax[i] = height[i+1];
17         continue
18     }
19     rightMax[i] = Math.max(rightMax[i+1], height[i+1])
20 }
21 console.timeEnd('max')
22 console.log(leftMax)
23 console.log(rightMax)
24 for (let i = 0; i <= height.length - 1; i++) {
25     if (i === 0 || i === height.length - 1) {
26         continue;
27     }
28     let currentHeight = height[i];
29     let leftHighest = leftMax[i];
30     let rightHighest = rightMax[i];
31     let lower = leftHighest > rightHighest ? rightHighest : leftHighest;
32     if (lower > currentHeight) {
33         sum += (lower - currentHeight);
34     }
35 }
36 return sum;
37 };

```

升级版：3D接雨水

要点：从普通接雨水的寻找左右侧最大值，增加寻找前后维度的最大值，再将四个值进行比较取与当前值的差值。



实现一个带并发控制的异步调度功能（Medium）

JS实现一个带并发限制的异步调度器Scheduler，保证同时运行的任务最多有两个。完善代码中Scheduler类，使得以下程序能正确输出

```

1  class Scheduler {
2    add(promiseCreator) { ... }
3    // ...
4  }
5  const timeout = (time) => new Promise(resolve => {
6    setTimeout(resolve, time)

```

```

7   })
8   const scheduler = new Scheduler()
9   const addTask = (time, order) => {
10    scheduler.add(() => timeout(time))
11    .then(() => console.log(order))
12  }
13  addTask(1000, '1')
14  addTask(500, '2')
15  addTask(300, '3')
16  addTask(400, '4')
17  // output: 2 3 1 4// 一开始, 1、2两个任务进入队列// 500ms时, 2完成, 输
18  // 出2, 任务3进队// 800ms时, 3完成, 输出3, 任务4进队// 1000ms时, 1完成, 输
19  // 出1// 1200ms时, 4完成, 输出4

```

参考答案

注意以下两点：

- 构建每个任务的 Promise，并缓存对应的 resolve 方法，以便完成执行后进行回调
- 需要不停的执行调试函数

```

1   class Scheduler {
2     concurrency = 2
3     running = 0
4     queue = []
5     add(task) {
6       return new Promise(resolve => {
7         this.queue.push({
8           taskGenerator: task,
9           resolve
10        })
11        this.schedule()
12      })
13    }
14    schedule() {
15      while (this.queue.length > 0 && this.running < this.concurrency) {
16        const curTask = this.queue.shift()
17        this.running += 1
18        curTask.taskGenerator().then(result => {
19          this.running -= 1
20          curTask.resolve(result)
21          this.schedule()
22        })
23      }
24    }
25  }

```

软件版本号对比（Easy）

versions是一个项目的版本号列表，因多人维护，不规则

```

1   var versions=['1.45.0','1.5','6','3.3.3.3.3.3']

```

要求从小到大排序，注意'1.45'比'1.5'大

```
1  var sorted=['1.5','1.45.0','3.3.3.3.3.3','6']
```

参考答案

```
1  var versions=['1.45.0', '1.5.1','1.5','6','3.3.3.3.3.3']
2
3  function sortVersion(arr) {
4      return arr.sort((a, b) => {
5          const arrA = a.split('.')
6          const arrB = b.split('.')
7          const len = arrA.length > arrB.length ? arrA.length : arrB.length;
8          for (let i = 0; i < len; i++) {
9              if (arrA[i] === undefined) {
10                 return -1
11             } else if (arrB[i] === undefined) {
12                 return 1
13             } else if (parseInt(arrA[i]) !== parseInt(arrB[i])) {
14                 continue
15             } else {
16                 return parseInt(arrA[i]) > parseInt(arrB[i]) ? 1 : -1
17             }
18         }
19     })
20 }
21
```

具体技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



实际问题类型

使用循环语句实现每隔1秒输出数组中的一个数字（Easy）

知识点



可以思考一下，一共有多少种方法可以实现

关键点：如何在某个位置保存临时数据

闭包

在一个作用域中可以访问另一个函数内部的局部变量的函数。

闭包的特性

1. 闭包可以访问到父级函数的变量
2. 访问到父级函数的变量不会销毁

参考答案：局部变量

```
1  function fun(arr) {
2      var len;
3      for (let i = 0, len = arr.length; i < len; i++) {
4          setTimeout(function() {
5              console.log(arr[i]);
6          }, i * 1000);
7      }
8  }
9
10 fun([1,2,3,4]);
11
12
```

参考答案：闭包

```
1  function fun(arr) {
2      var i, len;
3      for (i = 0, len = arr.length; i < len; i++) {
4          (function(i){
5              setTimeout(function() {
6                  console.log(arr[i]);
7              }, i * 1000);
8          })(i);
9      }
10 }
11
12 fun([1,2,3,4]);
```

参考答案：借助callee实现

```
1  function fun(arr) {
2      var i, len;
3      for (i = 0, len = arr.length; i < len; i++) {
```

```
4     var currentIndex = arr[i];
5     var fn = function () {
6         console.log(arguments.callee.currentIndex);
7     };
8     fn.currentIndex = currentIndex;
9     setTimeout(fn, currentIndex * 1000);
10 }
11 }
12
13 fun([1,2,3,4]);
```

电话拨号盘（Medium）

使用 js 和 css 实现一个电话拨号盘，要求如下：

- 包括12个按钮，从1至0，以及 # 和 *
- 按下某个数字或者字符时，对应的按钮高亮显示
- 可以通过上下左右箭头实现对高亮按钮的移动，且如果在第一行按向上，显相当于移动到最后一行，如果是最左侧按左箭头，相当于移动至最右列

参考答案

```
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>电话拨号盘</title>
8      <style>
9          body {
10              display: flex;
11              justify-content: center;
12              align-items: center;
13              height: 100vh;
14              margin: 0;
15              background-color: #f0f0f0;
16          }
17
18          .dial-pad {
19              display: grid;
20              grid-template-columns: repeat(3, 80px);
21              grid-gap: 10px;
22          }
23
24          button {
25              width: 80px;
26              height: 80px;
27              font-size: 24px;
28              border: none;
29              border-radius: 50%;
30              background-color: #fff;
31              box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
32              cursor: pointer;
33              transition: background-color 0.3s ease;
34          }
35
36          button:focus {
37              outline: none;
```

```
38     background-color: #4CAF50;
39     color: white;
40 }
41 </style>
42 </head>
43
44 <body>
45   <div class="dial-pad">
46     <button>1</button>
47     <button>2</button>
48     <button>3</button>
49     <button>4</button>
50     <button>5</button>
51     <button>6</button>
52     <button>7</button>
53     <button>8</button>
54     <button>9</button>
55     <button>*</button>
56     <button>0</button>
57     <button>#</button>
58   </div>
59   <script>
60     const buttons = document.querySelectorAll('.dial-pad button');
61     let currentIndex = 0;
62
63     // 初始化高亮第一个按钮
64     buttons[currentIndex].focus();
65
66     // 处理按钮点击事件
67     buttons.forEach((button, index) => {
68       button.addEventListener('click', () => {
69         currentIndex = index;
70         highlightButton(currentIndex);
71       });
72     });
73
74     // 处理键盘事件
75     document.addEventListener('keydown', (event) => {
76       switch (event.key) {
77         case 'ArrowUp':
78           currentIndex = (currentIndex - 3 + 12) % 12;
79           break;
80         case 'ArrowDown':
81           currentIndex = (currentIndex + 3) % 12;
82           break;
83         case 'ArrowLeft':
84           currentIndex = (currentIndex - 1 + 12) % 12;
85           break;
86         case 'ArrowRight':
87           currentIndex = (currentIndex + 1) % 12;
88           break;
89         default:
90           const key = parseInt(event.key, 10);
91           console.log('key', key, event.key);
92           if (isNaN(key)) {
93             return; // 如果不是方向键，不执行任何操作
94           }
95           if (key === 0) {
96             currentIndex = 10;
97           } else {
98             currentIndex = key - 1;
99           }
100         }
```

```
100         break;
101     }
102     console.log('currentIndex', currentIndex);
103     highlightButton(currentIndex);
104 });
105
106 // 高亮当前按钮
107 function highlightButton(index) {
108     buttons.forEach((button, i) => {
109         if (i === index) {
110             button.focus();
111         } else {
112             button.blur();
113         }
114     });
115 }
116 </script>
117 </body>
118
119 </html>
```

setTimeout / Promise 的执行问题（Medium，Hard）

请预测一下代码的输出：

```
1  console.log('stack [1]');
2  const p = Promise.resolve();
3  setTimeout(() => {
4      console.log("macro [2]")
5  }, 0);
6  setTimeout(() => console.log("macro [3]"), 1);
7
8  for(let i = 0; i < 3; i++) p.then(() => {
9      setTimeout(() => {
10         console.log('stack [4]')
11         setTimeout(() => console.log("macro [5]"), 0);
12         p.then(() => console.log('micro [6]'));
13     }, 0);
14     console.log("stack [7]");
15 });
16
17 console.log("macro [8]");
```

知识点

浏览器中的事件循环

1. 执行同步代码。
2. 执行一个同步任务（执行栈中没有就从任务队列中获取）。
3. 执行过程中如果遇到微任务，就将它添加到微任务的队列中。
4. 宏任务执行完毕后，立即执行当前微任务队列中的所有微任务（依次执行）。
5. 当前宏任务执行完毕，开始检查渲染，然后渲染线程接管进行渲染。
6. 渲染完毕后，JavaScript 线程继续接管，开始下一个循环。

参考答案

```
1  stack [1]
2  macro [8]
3
4  stack [7] stack [7] stack [7]
5
6  macro [2]
7
8  stack [4]
9  micro [6]
10 stack [4]
11 micro [6]
12 stack [4]
13 micro [6]
14
15 macro [3]
16
17 macro [5] macro [5] macro [5]
```

加强版

```
1  console.log('stack [1]');
2  const p = Promise.resolve();
3  setTimeout(() => {
4      console.log("stack [2]")
5      setTimeout(() => {
6          for(let i = 0; i < 3; i++) {
7              p.then(() => console.log('micro [2-1]'));
8          }
9      }, 0);
10     p.then(() => console.log('micro [2-2]'));
11 }, 0);
12 setTimeout(() => console.log("macro [3]"), 1);
13
14 for(let i = 0; i < 3; i++) p.then(() => {
15     setTimeout(() => {
16         console.log('stack [4]')
17         setTimeout(() => console.log("macro [5]"), 0);
18         p.then(() => console.log('micro [6]'));
19     }, 0);
20     console.log("stack [7]");
21 });
22
23 console.log("stack [8]");
```

答案

```
1  stack [1]
2  stack [8]
3  stack [7] stack [7] stack [7]
4  stack [2]
5  micro [2-2]
6  stack [4]
7  micro [6]
8  stack [4]
9  micro [6]
10 stack [4]
11 micro [6]
12 micro [2-1] micro [2-1] micro [2-1]
```


13 macro [5] macro [5] macro [5]

14 macro [3]

师资&课程内容

1. 师资：一线大厂/外企现役P8级高级专家亲自带；
2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

项目实战内容

github：<https://github.com/encode-studio-fe>

[目前端实战课分享汇总](#)（可咨询班主任要密码权限）

服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

近期报名学员案例

